

V-Trace: Final Project Report

Multi-Attribute Vehicle Re-Identification System

Gurneet Kaur Bhuller (A0354164E) · Group 21

Graduate Certificate in Pattern Recognition Systems · Practice Module

31 October 2026

Contents

Executive summary

1. Background and problem

2. Objectives and scope

3. System design

    3.1 Videos → sightings (`vtrace/video.py`)

    3.2 Camera details and real-world time (`vtrace/cameras.py`)

    3.3 Describing each sighting (`vtrace/features.py`, `vtrace/fastreid.py`)

    3.4 Description → ranked matches (`vtrace/search.py`)

    3.5 Selected match → where it was seen (`vtrace/reid.py`)

    3.6 Streamlit application (`app.py`)

4. Technique group mapping

5. Evaluation

    5.1 The Re-ID model on VeRi-776

    5.2 Demo cameras

6. Discussion and failure analysis

7. Limitations and future work

8. Conclusion

References

Appendix A – Running the system

Appendix B – Deliverables

AI declaration

Executive summary

V-Trace is a runnable pattern-recognition system for **vehicle search and cross-camera re-identification (Re-ID)**. The user loads short dash-cam or traffic-camera videos, either the demo videos already on the platform or new uploads, and V-Trace processes each video into *sightings*: every vehicle is detected, followed while it stays in view, and described. The user then describes the vehicle they are looking for (colour, type, full or partial licence plate, distinctive features, free text). V-Trace returns a **ranked list of best matches**. When the user selects one, V-Trace shows **every camera the vehicle was seen in, with the camera's location, the real-world date and time of the sighting and one frame from it**.

The system is built entirely from **published, pretrained models**. Nothing is trained for this project: YOLO11 detects vehicles, ByteTrack follows them, CLIP scores the description zero-shot, EasyOCR reads plates, and the **FastReID VeRi-776 model** released by the VeRi-776 authors' team (JDAI-CV) matches vehicles across cameras.

Key results:

- **The Re-ID model as published, on the official VeRi-776 split:** Rank-1 97.0%, Rank-5 98.6%, mAP 81.6% (1,678 queries, 11,579 gallery images; FastReID reports 97.0% / 81.9%).
- **On the demo cameras** (real street footage, 187 sightings in four cameras): the correct sighting of the same vehicle in another camera is ranked first for 36% of queries (Rank-5 58%, mAP 38%). At the default threshold, **81% of the cameras V-Trace reports are correct** and it finds 32% of the cameras the vehicle really passed through.
- One Streamlit application with an admin sign-in covers the whole flow: videos → describe → ranked matches → select → where it was seen.

1. Background and problem

Vehicle re-identification means deciding whether a vehicle seen by one camera is the same vehicle seen by another. A licence plate is the strongest single identifier, but in traffic footage it is often unreadable: too small, blurred, occluded, badly lit or facing away. Investigators also rarely start with an image. They start with a description, such as "a red sedan with a spoiler, plate starting SGX12".

V-Trace therefore works in two steps. First, a **description search** turns what the user knows into a ranked list of sightings. Second, once the user has picked the right sighting, **image-based Re-ID** finds the same vehicle in every other camera, without needing the plate.

2. Objectives and scope

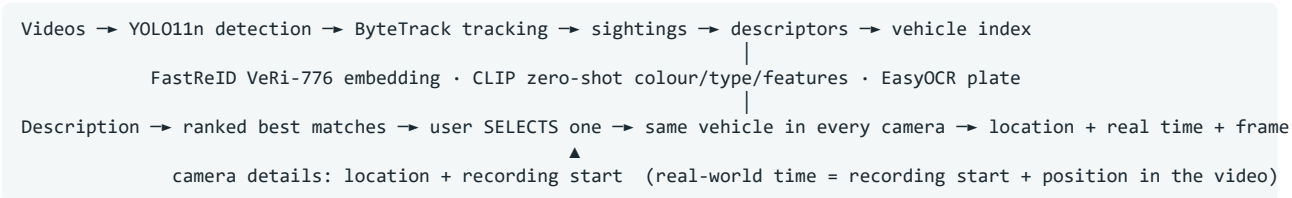
Objective. Build a basic, working vehicle Re-ID platform that:

- offers two inputs: demo videos already on the platform, or new short uploaded videos (one video = one camera);
- processes each video and groups the frames in which one vehicle is seen continuously into one **sighting** (e.g. a red sedan visible for 4 s in camera 1);
- takes a description of the target (colour, type, plate, distinctive features, free text) and returns ranked best matches;
- after the user selects a match, shows every camera in which that vehicle was seen, with the camera's location, the real-world time of the sighting and one frame from it. Cameras may be seconds, minutes or hours apart in time.

In scope	Out of scope
Short uploaded or bundled videos; detection; tracking; sightings	Live CCTV networks
Colour / type / plate / distinctive-feature description search	Vehicle-owner records, facial recognition
Cross-camera Re-ID with a pretrained model; ranked results with frames and timestamps	Training detection or Re-ID networks

The output is always a **ranked list for a person to review**, never an identity claim.

3. System design



Step	Model (pretrained, used as published)	Source
Detect vehicles	YOLO11n, COCO classes car / motorcycle / bus / truck	Ultralytics
Follow a vehicle while it is in view	ByteTrack	Ultralytics
Colour, type, distinctive features, free text	CLIP ViT-B/32, zero-shot	OpenAI (open_clip)
Licence plate	EasyOCR	JaidedAI
Same vehicle in another camera	FastReID SBS(R50-ibn), trained on VeRi-776	JDAI-CV/fast-reid

3.1 Videos → sightings (`vtrace/video.py`)

Every 3rd frame (every 5th in *Fast processing*) goes through YOLO11n; ByteTrack links the detections of one vehicle over time. One track is one **sighting**: the vehicle's camera, its first and last time, its boxes, four good crops spread over time, and one full frame with the vehicle boxed (the frame where its crop is largest and sharpest). Tracks shorter than 4 detections are dropped as noise.

3.2 Camera details and real-world time (`vtrace/cameras.py`)

A video file only knows positions in the video (00:40.80), not when or where it was recorded. Each camera therefore has **details**: a location (place name, optional latitude/longitude) and the **recording start** (date and time). A sighting's real-world time is the recording start plus its position in the video, so two cameras can be seconds, minutes or hours apart, and the route is ordered by real-world time. For uploads the user enters the details next to each video; all details can be edited on the *Cameras* page and are saved with the index.

The demo videos carry no location or clock metadata, so their details are **curated** (Table 1): camA–camC are zones of one recording, so they share one clock and are placed a few tens of metres apart along one street; camD, a different road, is given a recording start 2.5 minutes later.

Camera	Location (curated)	Recording start (curated)
camA_west	Slovenska cesta, west of the crossing · Ljubljana	Wed 28 Oct 2026, 08:15:00
camB_centre	Slovenska cesta, at the crossing · Ljubljana	Wed 28 Oct 2026, 08:15:00
camC_east	Slovenska cesta, east of the crossing · Ljubljana	Wed 28 Oct 2026, 08:15:00
camD_other_road	Celovška cesta · Ljubljana	Wed 28 Oct 2026, 08:17:30

3.3 Describing each sighting (`vtrace/features.py`, `vtrace/fastreid.py`)

- **Re-ID embedding.** Each crop goes through the FastReID VeRi-776 network (ResNet-50 with IBN-a and non-local blocks, GeM pooling, BN-neck; 256×256 input); the four 2048-d embeddings are averaged and L2-normalised. FastReID is a large training framework, so `vtrace/fastreid.py` only re-declares the network so that the published checkpoint loads with plain PyTorch; the weights are used unchanged.
- **Colour and type.** CLIP compares the averaged crop embedding with prompts such as "a photo of a red car" or "a photo of a sedan car" (ten VeRi colours, nine VeRi types) and keeps the softmax probabilities.
- **Distinctive features and free text.** CLIP scores prompts such as "a photo of a car with a rear spoiler" against the neutral "a photo of a car".
- **Plate.** EasyOCR reads the lower part of the two largest crops; a plate is kept only if it is confidently read.

3.4 Description → ranked matches (`vtrace/search.py`)

Each field the user fills in gives every sighting a 0–1 score: colour and type (the CLIP probability of the requested value, relative to the most likely value), plate (wildcard-aware similarity; `?` = unknown character; a sighting whose plate could not be read gets a neutral 0.3), distinctive features and free text (CLIP prompt scores). The final score is the weighted mean over **only the fields provided** (plate 0.35, colour 0.30, type 0.25, features 0.10, text 0.10), so a missing field never counts against a vehicle. The 12 best sightings are shown.

3.5 Selected match → where it was seen (`vtrace/reid.py`)

The similarity of two sightings is the cosine of their FastReID embeddings. For the selected sighting, V-Trace checks every other camera and takes that camera's most similar sighting. It is reported as the same vehicle only if

- its similarity is at least **0.40** (chosen from the VeRi-776 score distributions, Section 5.1);
- it beats the camera's second-best sighting by at least **0.08**. This is a ratio test (as in Lowe's SIFT matching): when two vehicles are almost equally similar, the answer is ambiguous and is left to the user;
- with the real-world time limit on, it is within **10 s** (adjustable from 5 s to 12 h) of the selected sighting in real-world time. Cameras whose recording start is unknown are not time-limited.

Each reported sighting is shown with its camera, location, real-world date and time span, position in the video, a crop and the full frame; the route lists them in real-world time order with the time between cameras (e.g. "+1.2 s", "+2 min 30 s"), and a map shows the camera locations. For cameras with no confident match, the three closest vehicles are shown so the user can judge. All three settings can be changed in the app; for uploads the time limit starts off, in case the cameras' clocks are not set.

3.6 Streamlit application (`app.py`)

An admin sign-in (credentials in Streamlit secrets, compared in constant time) protects the app. The sidebar chooses **Demo videos** or **Upload new videos** (with a *Fast processing* switch) and lists the processed cameras. The Search page has three steps: *1 Describe the vehicle*, *2 Best matches* (cards with crop, camera, time span, colour/type, plate, score and **Select**), *3 Where this vehicle was seen* (route with real-world times, one row per sighting with location, date/time, crop and frame, and a map). The *Cameras* page lists and edits every camera's location and recording start; uploads ask for them next to each video. A *How it works* page lists the models and the benchmark numbers. The app opens in dark mode with the V-Trace logo and palette. The demo cameras are bundled pre-processed (`demo_bundle/`) so the hosted app loads them instantly.

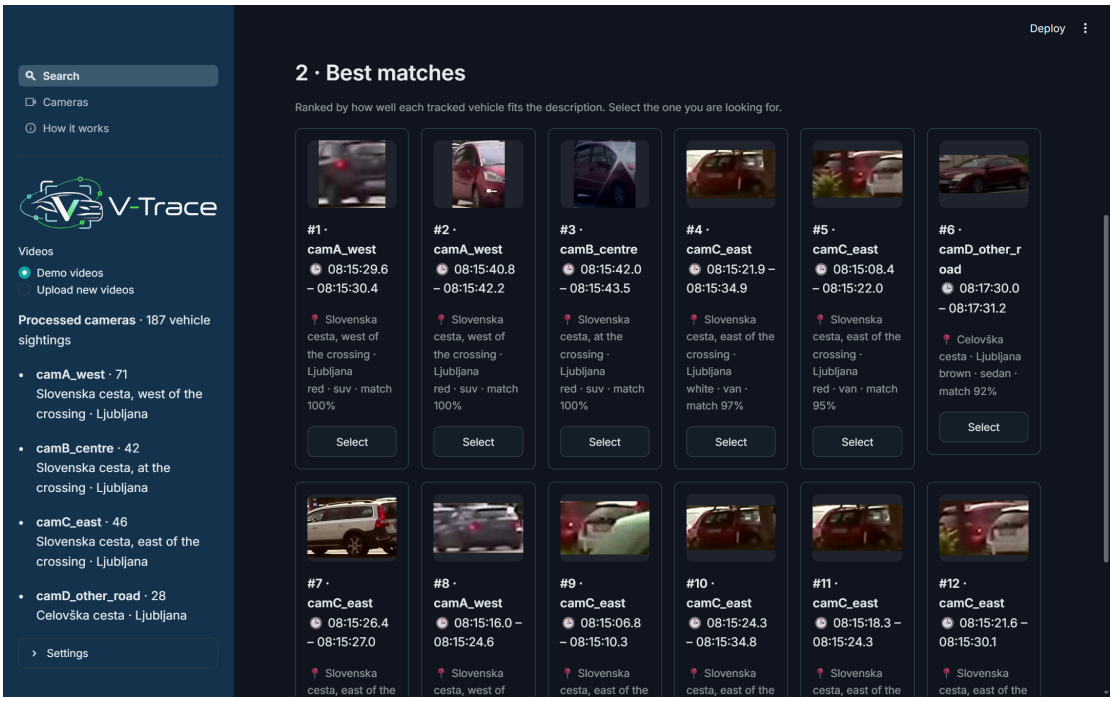


Figure 1 – describe the vehicle and pick it from the ranked best matches

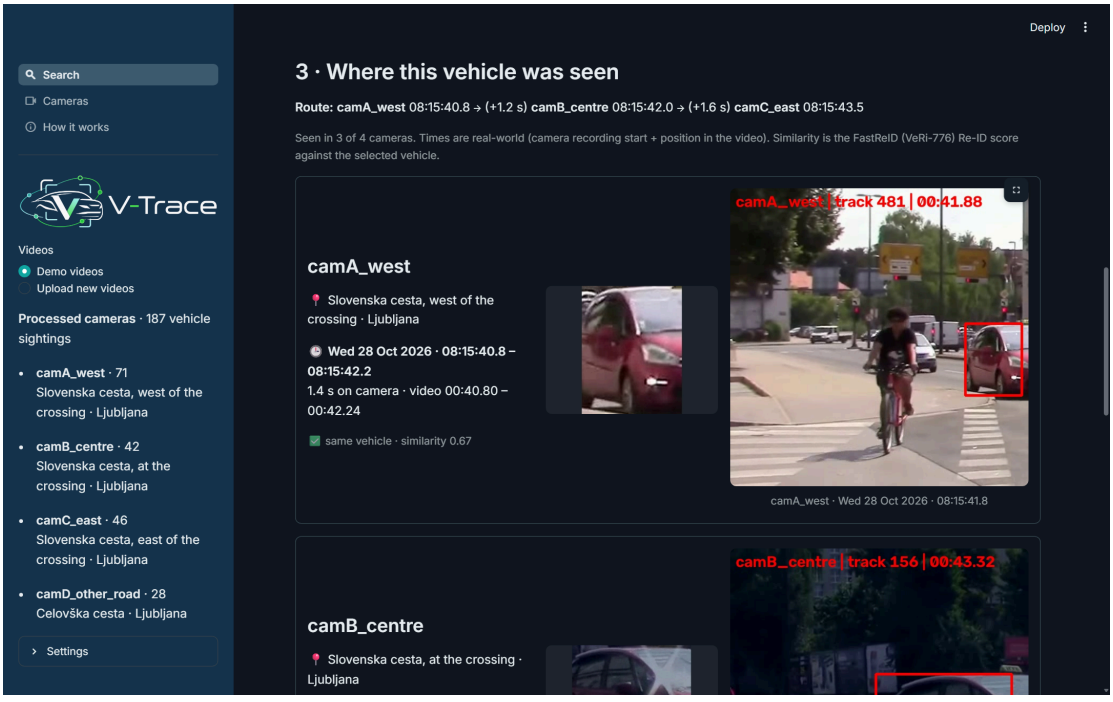


Figure 2 – where the selected vehicle was seen: camera, location, real-world time and a frame per sighting

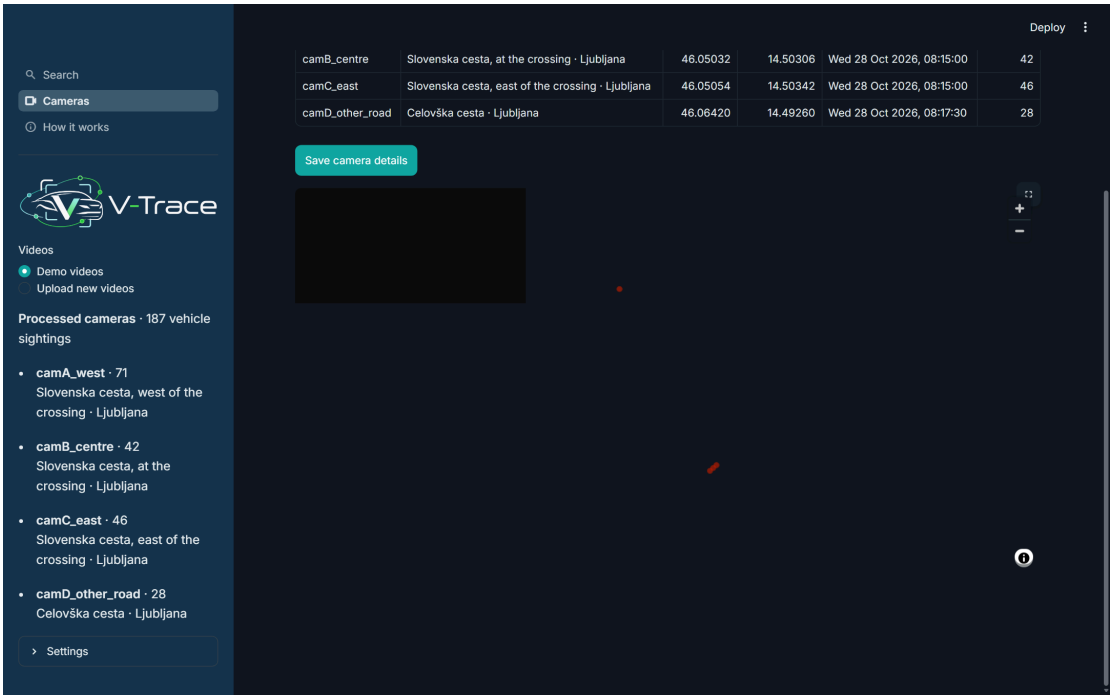


Figure 3 – the Cameras page: location and recording start of every camera

4. Technique group mapping

Technique group	In V-Trace
Machine / deep learning	Pretrained deep models: YOLO11 detector, FastReID Re-ID network, CLIP, EasyOCR
Supervised learning	The FastReID model is a supervised metric-learning model (trained by its authors on VeRi-776 identities with softmax + triplet losses); we apply it to new footage
Hybrid / multi-evidence	Description search combines colour, type, plate, feature and free-text evidence with weights that adapt to the fields given
Intelligent sensing / sense-making	Raw video → detections → tracks → sightings → descriptors → ranked matches → selected vehicle → sightings across cameras

5. Evaluation

5.1 The Re-ID model on VeRi-776

VeRi-776 (Liu et al., 2016) has more than 50,000 images of 776 vehicles from 20 cameras. We ran the published checkpoint on the **official split**: each of the 1,678 query images is ranked against the 11,579 test images; gallery images of the same vehicle from the same camera are ignored (standard protocol). Nothing was trained.

Metric	Our run	FastReID model zoo
Rank-1	97.0%	97.0%
Rank-5	98.6%	–
mAP	81.6%	81.9%

Choosing the threshold. Same-vehicle pairs from different cameras have a median similarity of 0.62 (10th percentile 0.39), while 99% of different-vehicle pairs score below 0.38. At the default threshold of 0.40, 89% of same-vehicle pairs and 0.8% of different-vehicle pairs pass.

5.2 Demo cameras

The demo uses a 46 s, 1080p street recording from Ljubljana (Wikimedia Commons, CC BY 3.0). The frame is cut into **three non-overlapping camera zones along the road** (camA_west, camB_centre, camC_east), each with a different exposure and colour cast, so a car on the near lane passes A → B → C and a car on the far lane C → B → A. A fourth camera (camD_other_road) is a different road whose vehicles never appear in A–C.

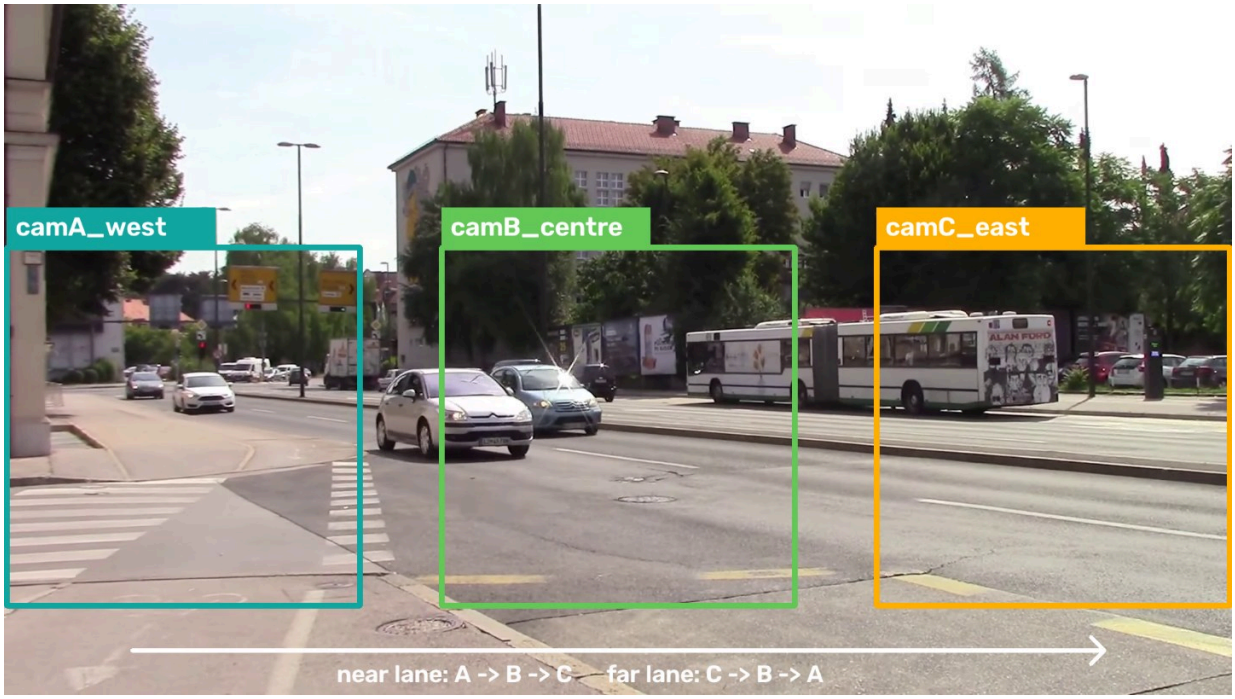


Figure 4 – the three demo camera zones cut from one street recording

Processing gives 187 sightings (camA_west 71, camB_centre 42, camC_east 46, camD_other_road 28). Because the three street cameras come from one recording, tracking the uncropped clip once at high resolution and mapping every zone sighting back onto it by box overlap gives a reference identity per sighting without manual labelling (*pseudo* ground truth: it inherits the reference tracker's own mistakes). 19 vehicles appear in two or more cameras and 7 in all three.

Measure (`scripts/demo_eval.py`)	Value
Queries (sightings whose vehicle also appears in another camera)	55
Rank-1 / Rank-5 / mAP, other cameras only	36% / 58% / 38%
Cameras reported that are correct (precision), with / without the 10 s real-world window	81% / 53%
Cameras passed through that are reported (recall), with / without the window	32% / 22%
Queries wrongly reported in the distractor camera camD, with / without the window	0% / 16%

Why the decision rule matters. With a similarity threshold alone (every sighting above it reported), only about 40% of the reported cameras were correct and more than half of the queries were wrongly placed on the distractor road. Keeping only each camera's best sighting, adding the ratio test and the time window raised precision to 81% without any training. The cost is recall: for cameras without a confident match the user sees the closest candidates instead. Part of the gain with the window comes from the curated clocks: camD's recording starts 2.5 minutes after camA–camC, so a 10 s window rules it out completely. The numbers without the window show what the Re-ID model and the ratio test achieve on their own. | Sightings that follow one vehicle only (tracking purity ≥ 95%) | 61% |

6. Discussion and failure analysis

Benchmark vs real footage. The model is excellent on VeRi-776, the data it was trained for, and clearly weaker on the demo footage, which comes from a different city, camera height and resolution: many vehicles are small (60–150 px) and seen from very different angles in neighbouring zones. This domain gap is the main reason for keeping a person in the loop: V-Trace shows its evidence (frames, time spans, scores, and the closest candidate where nothing passed the threshold) rather than making an identity claim.

Observed failure modes

- **Scale and viewpoint change:** a vehicle seen small and front-on in one zone and large and side-on in another gives lower similarity and can fall below the threshold.
- **Look-alike vehicles:** silver and white compact cars are common; a similar car can pass the threshold.
- **Track fragmentation and switches:** buses occlude cars in the far lane and the zone edges cut vehicles, splitting one vehicle into several sightings; occasionally a track switches between two cars.
- **Attribute errors:** CLIP zero-shot confuses estate, hatchback and MPV, and silver vs grey vs white.
- **No plates:** none were readable at this camera distance, which is exactly why image-based Re-ID is needed.

7. Limitations and future work

- The demo cameras are zones of one recording, not separate physical cameras; a multi-camera dataset such as CityFlow would be a stronger test.
- The demo cameras' locations and clocks are curated; real deployments would read them from the camera system, and learned travel times between camera locations would replace the single fixed time window.
- A dedicated plate detector before OCR would make plate evidence usable at longer distances.
- The pipeline reads files frame by frame, so live streams are a small change.

8. Conclusion

V-Trace shows that a useful vehicle search and Re-ID platform can be assembled from published pretrained models: YOLO11 + ByteTrack turn video into sightings, CLIP and EasyOCR make them searchable by description, and the FastReID VeRi-776 model links the selected vehicle across cameras. The model reproduces its published accuracy on VeRi-776 (mAP 81.6%), and on real street footage it reports the right cameras with 81% precision, while the frames and time spans let a person check every result.

References

- Liu, X., Liu, W., Mei, T., & Ma, H. (2016). A Deep Learning-Based Approach to Progressive Vehicle Re-identification for Urban Surveillance (VeRi-776). ECCV.
- He, L., Liao, X., Liu, W., Liu, X., Cheng, P., & Mei, T. (2020). FastReID: A Pytorch Toolbox for General Instance Re-identification. arXiv:2006.02631. https://github.com/JDAI-CV/fast-reid
- Pan, X., Luo, P., Shi, J., & Tang, X. (2018). Two at Once: Enhancing Learning and Generalization Capacities via IBN-Net. ECCV.
- Zhang, Y. et al. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. ECCV.
- Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML.
- Jocher, G. et al. Ultralytics YOLO11 and multi-object tracking documentation. https://docs.ultralytics.com/
- JaidedAI. EasyOCR. https://github.com/JaidedAI/EasyOCR
- Sounds of Changes. Road traffic in Ljubljana (video), Wikimedia Commons, CC BY 3.0.

Appendix A – Running the system

1. Install (Windows, Python 3.11): `python -m venv .venv`, then `.venv\Scripts\pip install -r requirements.txt`.
2. Create the admin account: copy `.streamlit/secrets.toml.example` to `.streamlit/secrets.toml` and set a username and password.
3. Run `.venv\Scripts\python -m streamlit run app.py`, open http://localhost:8501 and sign in. The pretrained weights download on first use.
4. Benchmark: download VeRi-776 from its authors, then run `python scripts/eval_veri.py`; demo evaluation: `python scripts/demo_eval.py`.

Step-by-step demo scenarios and a guide for assessors are in `docs/DEMO_GUIDE.md`.

Appendix B – Deliverables

Deliverable	Location
Runnable system (source code, tests)	`app.py`, `vtrace/`, `scripts/`, `tests/`
Pretrained models	downloaded on first use (FastReID VeRi-776 checkpoint into `models/`)
Datasets	demo cameras (`demo_bundle/`, built by `scripts/prepare_demo_clips.py`); VeRi-776 by request from its authors
Results	`results/veri_eval.json`, `results/demo_eval.json`
Final report	this document
Presentation slides	`V-Trace_First_Presentation.pdf`, `V-Trace_Final_Presentation.pptx`
Demo guide	`docs/DEMO_GUIDE.md`

AI declaration

In preparing this project, AI-based tools including Claude, ChatGPT and Gemini were used to support drafting and refining text, organising content, implementation and general research support. All ideas, analysis, technical design decisions and conclusions are the work of the project team.